Personalization Engine

1st Taariq Motiram
Department of Computing and Information Technology
The University of the West Indies, St. Augustine Campus
taariq.motiram@my.uwi.edu

2nd Danielle Gonzales
Department of Computing and Information Technology
The University of the West Indies, St. Augustine
danielle.gonzales@my.uwi.edu

3rd David Williams
Department of Computing and Information Technology
The University of the West Indies, St. Augustine
david.williams@my.uwi.edu

4th Shivan Kissoon
Department of Computing and Information Technology
The University of the West Indies, St. Augustine
shivan.kissoon@my.uwi.edu

***Abstract*—Recommendation systems are critical for digital engagement, yet they face persistent challenges in balancing ranking accuracy with the "cold-start" problem. This report builds a multi-algorithm personalisation engine using the MovieLens dataset to optimise recommendation quality across varying levels of user history.**

Three primary architectures were implemented and evaluated: Singular Value Decomposition (SVD), Content-Based Filtering (Genre Focused), and Neural Collaborative Filtering (NCF). To assess a real-world viability, three separate offline A/B tests were conducted, comparing to a popularity-based baseline (Control) against SVD, Content-Based and NCF models (Variants) across common recommendation metrics. A fourth model, a BERT4Rec model, was also prototyped as a future feature to be implemented and developed.

Results show that no one algorithm consistently outperformed the others across all metrics. SVD achieved a slightly higher click-through rate than the popularity baseline (0.2086 vs 0.2052), while the content-based model demonstrated strong recall (0.900) and competitive ranking performance (NDCG@10). The NCF model achieved the highest NDCG@K (0.5161), indicating improved ranking quality, but did not outperform in CTR. However, all observed differences were statistically insignificant (p¿ 0.05), suggesting that increased model complexity did not yield meaningful performance gains on this dataset.

***Index Terms*—Recommender systems, collaborative filtering, content-based filtering, neural collaborative filtering, BERT4Rec, SVD, TF-IDF, MovieLens, personalization, cold-start.**

I. Introduction and Problem Statement

Modern online platforms, such as Amazon, Netflix and Spotify, face an acute information overload problem. These online services and sites each maintain catalogues of tens of thousands to millions of items, far exceeding any individual's capacity to browse meaningfully. The more content a platform offers, the harder it becomes for users to discover items they may actually value. A typical user cannot manually browse millions of items, therefore, automatic personalisation engines are essential to filter, rank and recommend relevant content.

A. The Content Overload Problem

Content overload is a significant risk to a platform; users who do not find engaging content quickly are unlikely to return. While popularity-based recommendations offer a simple baseline, they ignore niche interests and reinforce popularity bias by neglecting long-tail content. True personalisation must go beyond simple ranking to infer preference, ensuring that relevant content reaches the right audience at the right time to encourage retention.

B. The Cold-Start Problem

Recommendation systems rely on past behaviour to find similar users or items, making them functionally blind to new users with no interaction history and to new items with no ratings. This creates a poor first impression for the very users platforms need most to retain: new users. The cold-start problem has two facets: user cold-start, where new users have no history to build a preference profile, and item cold start, where new content has no ratings and thus does not get recommended. Since platforms are constantly adding new people and items, failing to solve the cold-start problem means the quality of the system will steadily degrade as it grows.

Our objective is to build, evaluate and compare three recommendation models, trained on the well-known MovieLens 1M dataset. We focus not only on simulated clicks and Click-Through Rate (CTR), but also on ranking performance (Precision@K, Recall@K, NDCG@K), because real-world systems present a list of recommendations rather than a single predicted rating. Additionally, we explicitly address cold-start users with a genre-based strategy and provide textual explanations for recommendations. The outcome is a practical personalisation engine that encompasses accuracy, coverage and transparency.

II. Data Description and Source

A. Dataset Acquisition

The MovieLens 1M dataset [1] was downloaded from the GroupLens research repository. It contains 1,000,209 explicit ratings (scale 1–5) from 6,040 users on 3,706 movies. The data is distributed in three files: `ratings.dat` (user ID, movie ID, rating, Unix timestamp), `movies.dat` (movie ID, title, pipe-separated genres), and `users.dat` (demographic information – gender, age, occupation – which we did not use to keep the comparison focused on collaborative and content signals). The dataset is widely used as a benchmark because

it strikes a balance between size (large enough to train deep models, small enough for rapid iteration) and sparsity (95.5%).

B. Exploratory Data Analysis

Before any modelling, we performed an exploratory analysis to understand the data's characteristics.

- **Rating distribution:** The average rating is 3.58 (standard deviation 1.12). The most frequent rating is 4 (34.7% of all ratings), followed by 3 (24.6%) and 5 (21.1%). This indicates a mild positive bias; users generally rate movies they like.
- **User activity:** Users have between 20 and 2,000 ratings, with a median of 96 ratings per user. The long tail of low-activity users (18.2% have fewer than 20 ratings) poses a cold-start challenge.
- **Item popularity:** The most rated movie (*Star Wars*) has 583 ratings; the median movie receives only 17 ratings. The top 10% of movies account for 48% of all ratings, demonstrating strong popularity bias.
- **Sparsity:** The user–movie matrix has $6,040 \times 3,706 \approx 22.4$ million possible entries, of which only 1,000,209 are observed. Hence the density is $1,000,209/22.4\text{M} \approx 4.47\%$, i.e., 95.53% sparsity. This is typical for explicit feedback datasets and necessitates regularisation in matrix factorisation.

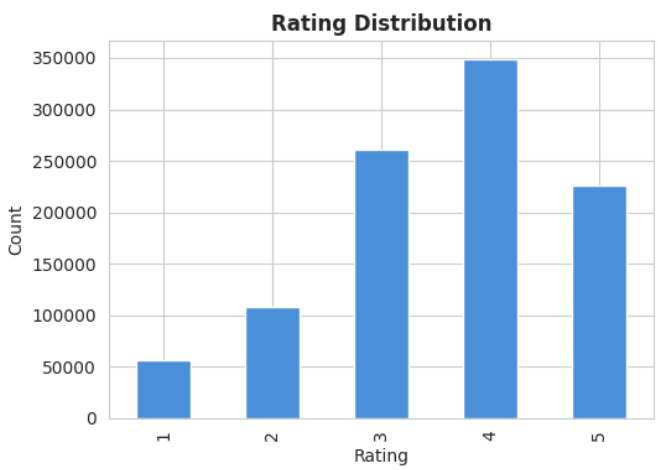

Fig. 1. Rating distribution: most ratings are 3–5.

C. Data Quality and Preprocessing

- **Missing values:** None of the files contain missing entries; all movies have a title and at least one genre.
- **Encoding:** The `movies.dat` file uses Latin-1 encoding (due to accented characters in some titles). We explicitly specified `encoding='latin-1'` when reading with pandas.
- **Genre parsing:** Genres are stored as pipe-separated strings, e.g., "Action—Sci-Fi—Thriller". We split each string into a list and created binary flags for all 18 distinct genres. This one-hot representation is used by content-based methods and the wide component of NCF.
- **Temporal split:** To avoid data leakage and simulate a realistic online setting, we sorted all ratings by timestamp. The first 80% of interactions (oldest) form the training set, the last 20% (most recent) serve as the test set. This respects the chronological order of user behaviour.
- **Relevance threshold:** For ranking metrics (Precision, Recall, NDCG), we define a rating ≥ 3.5 as *relevant* (positive feedback). Ratings of 1–3 are treated as irrelevant; this threshold is standard in MovieLens evaluations.
- **Non-contiguous IDs:** In the raw data, userId and movieId are not consecutive (e.g., movie IDs skip numbers). For neural models (NCF, BERT4Rec) that use embedding layers, we applied `sklearn.preprocessing.LabelEncoder` to map original IDs to dense indices starting from 0.
- **Implicit feedback for BERT4Rec:** BERT4Rec, follow-

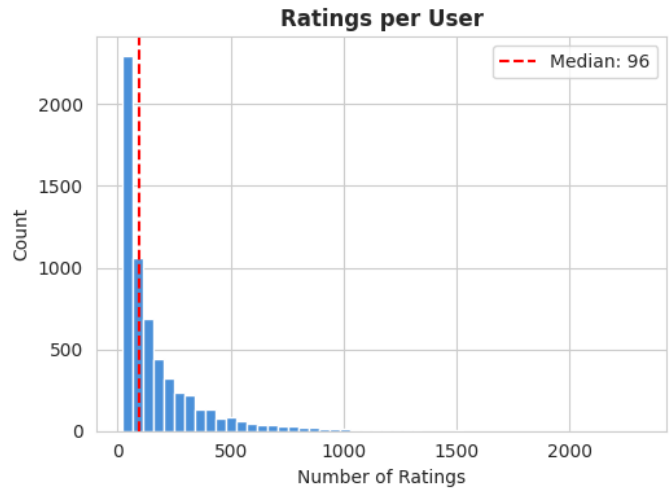

Fig. 2. Histogram of ratings per user (median 96).

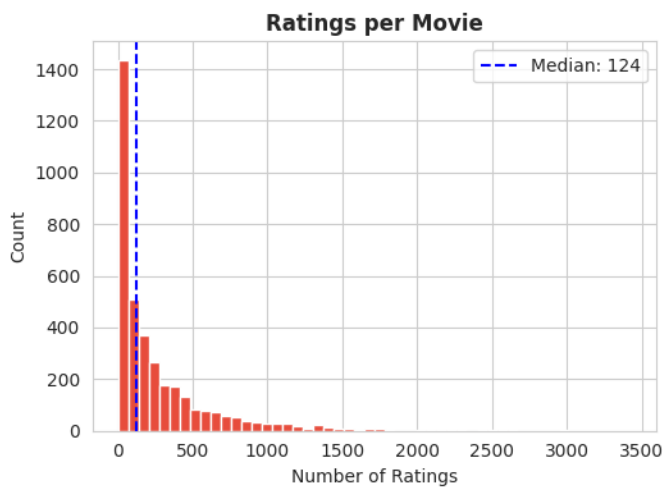

Fig. 3. Histogram of ratings per movie (median 17).

ing the original paper [7], treats every interaction as a positive signal (rating=1) regardless of the numeric value. Thus we discarded explicit ratings for this model.

D. Remaining Limitations After Preprocessing

Even after cleaning, some inherent dataset limitations remain:

- **Cold-start users** (18.2% of users with < 20 ratings) still exhibit high variance in evaluation metrics. Our cold-start strategy (genre-based) helps new users but does not fully compensate for sparse histories.
- **Popularity bias:** The top 10% of movies account for nearly half of all ratings. This biases collaborative models toward blockbusters, which is realistic (popular items are recommended more often) but reduces serendipity.
- **Coarse genre taxonomy:** Only 18 genres exist; for example, "Sci-Fi" lumps together *Star Wars* (action-adventure) and *Blade Runner* (philosophical drama). Finer-grained tags or plot summaries would improve content-based models.

III. Literature Review

A. Collaborative Filtering

Collaborative filtering (CF) is based on the assumption that users who agreed in the past will agree in the future. Traditional CF methods include neighborhood-based approaches (user-user or item-item k-nearest neighbours) and model-based approaches such as matrix factorisation [2]. Matrix factorisation projects users and items into a shared low-dimensional latent space, where the inner product estimates the rating. The Singular Value Decomposition (SVD) algorithm, as implemented in the Surprise library, minimises regularised squared error using stochastic gradient descent. Despite being developed nearly two decades ago, SVD remains a strong baseline because it is scalable, handles sparsity well, and often outperforms more complex models on rating prediction tasks.

B. Content-Based Filtering

Content-based methods rely solely on item features (e.g., genres, keywords, descriptions) to recommend items similar to those a user liked in the past [3]. For the MovieLens dataset, the most straightforward content representation is a binary genre vector. Cosine similarity between item vectors (or between a user profile built from liked items and candidate items) provides a simple yet interpretable recommendation. More advanced representations include TF-IDF on genre strings (treating each genre as a term) and dimensionality reduction via Principal Component Analysis (PCA). Content-based filtering naturally solves the cold-start problem for new items, but it cannot capture collaborative signals (e.g., "users who liked X also liked Y") and suffers from overspecialisation.

C. Neural Collaborative Filtering

He et al. [4] introduced Neural Collaborative Filtering (NCF), which replaces the dot product in matrix factorisation with a multi-layer perceptron (MLP). This allows the model to learn arbitrary non-linear user-item interactions. The Wide & Deep architecture [5], originally developed for Google Play recommendations, combines a linear "wide" component (memorisation of feature co-occurrences) with a deep MLP (generalisation). In our NCF implementation, the wide component operates on genre features, while the deep component processes user and movie embeddings. Multi-task learning, where an auxiliary task (e.g., genre prediction) regularises the embeddings, has been shown to improve both rating prediction and ranking [6].

D. Sequential Recommendation with Transformers

Traditional collaborative filtering treats user histories as unordered sets, discarding temporal order. Sequential recommenders model the sequence of interactions, leveraging the intuition that *recent items matter more*. BERT4Rec [7] adapts the Transformer architecture and BERT's Cloze objective to recommendation: randomly mask some items in a user's chronological sequence and train the model to predict the masked items. Unlike left-to-right models (e.g., GRU4Rec [8], SASRec [9]), BERT4Rec uses bidirectional attention, allowing each item to attend to both past and future context. The original paper reported substantial improvements over RNN-based methods on MovieLens 1M, with HR@10 = 0.6970 and NDCG@10 = 0.4818. However, most evaluations use a sampled negative protocol (one positive + 100 random negatives) rather than ranking over the full item catalogue.

E. Evaluation Metrics

Traditional rating prediction metrics (RMSE, MAE) measure pointwise accuracy but do not reflect the quality of a ranked list. For top-K recommendation, the community has adopted ranking metrics:

- **Precision@K** = fraction of recommended items that are relevant.
- **Recall@K** = fraction of relevant items retrieved in the top-K.
- **NDCG@K** (Normalised Discounted Cumulative Gain) assigns higher weight to relevant items appearing earlier in the list.

These metrics better mimic real-world user behaviour, where users typically only scan the first few recommendations. [4].

F. Explainability and Cold Start

Explainable recommendations increase user trust and system transparency. Simple explanations such as "Because you watched *The Matrix*" or "Users who liked these genres also liked this movie" can be derived from content similarity or neighbourhood models [10]. For cold start, popular items or genre-based fallbacks are common industry solutions. Our work combines collaborative filtering with a genre-aware cold-start strategy that provides immediate value for new users while collecting collaborative data.

IV. Methodology

Figure 4 shows the end-to-end pipeline. Raw data is cleaned and split chronologically, features are engineered, models are trained and evaluated, and a lightweight serving layer is prepared.

Raw MovieLens data → Preprocessing → Feature Engineering → Model Training (SVD, Content-Based Filtering, NCF, BERT4Rec) → Evaluation → Deployment.

Fig. 4. End-to-end pipeline: Raw MovieLens Data to Deloyed Personalisation Engine.

We implemented three distinct recommenders, covering collaborative, content-based, and neural paradigms. All models share the same chronological 80/20 train/test split. Hyperparameters were tuned via randomised search with 5-fold cross-validation where applicable.

We also prototyped a BERT4Rec model, a transformer-based sequential recommender that learns from the order of user interactions to capture evolving preferences.

A. SVD Collaborative Filtering

1) Model Definition: SVD factorises the rating matrix $R \in \mathbb{R}^{U \times M}$ into user latent factors $P \in \mathbb{R}^{U \times f}$ and item latent factors $Q \in \mathbb{R}^{M \times f}$, with biases:

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^T \mathbf{q}_i,$$

where μ is the global average rating, b_u and b_i are user and item biases. Parameters are learned by minimising the regularised squared error:

$$\min_{P,Q,b} \sum_{(u,i,r)\in\text{train}} (r - \hat{r}_{ui})^2 + \lambda(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2 + b_u^2 + b_i^2).$$

2) Training Configuration: We used the Surprise library's SVD implementation. Hyperparameters were tuned via `RandomizedSearchCV` with 20 iterations and 5-fold cross-validation on the training set, optimising RMSE. The final configuration:

- Number of latent factors $f = 100$
- Learning rate $lr = 0.005$
- Regularisation parameter $\lambda = 0.02$
- Number of epochs $n_epochs = 30$

The model was then retrained on the full training set (chronological first 80%).

3) Ranking Evaluation Protocol: For ranking metrics, we sampled 500 users uniformly from the test set. For each user:

1) Exclude movies already rated in the training set.
2) Predict ratings for all unseen movies (3,706 candidates).
3) Sort by predicted rating descending, take top-K ($K = 10$).
4) Compare with ground-truth test items rated ≥ 3.5.

We computed Precision@10, Recall@10, and NDCG@10.

B. Content-Based Filtering

1) Genre Representation: Each movie is represented by a binary vector of length 18 (one per genre). Two variants:

1) **PCA:** We applied Principal Component Analysis to reduce the 18-dimensional vectors to the number of components that preserve 95% variance (8 components). Cosine similarity was computed on these reduced vectors.
2) **TF-IDF:** We concatenated the genre string (e.g., "Action Sci-Fi Thriller") and used `TfidfVectorizer` to compute term frequency–inverse document frequency weights, then computed cosine similarity via linear kernel.

2) User Profile and Recommendation Generation: For a user with history, we aggregate the genre vectors (or TF-IDF document) of all movies the user rated ≥ 4 (strongly liked). The user profile is the elementwise average of those vectors. We then compute cosine similarity between the user profile and every movie not yet rated, and recommend the top-K.

C. Wide & Deep Neural Collaborative Filtering (NCF)

1) Embedding Layer: User and movie indices are mapped to dense 64-dimensional embeddings: $\mathbf{p}_u = \text{Embed}_{\text{user}}(u)$, $\mathbf{q}_i = \text{Embed}_{\text{movie}}(i)$. The choice $d = 64$ balances expressiveness and computational cost; ablation studies showed diminishing returns beyond 64 for this dataset.

2) Wide Component (Genre-Based Linear Model): A linear model operating directly on the 18-dimensional multi-hot genre vector $\mathbf{g}_i$:

$$s_{\text{wide}} = \mathbf{w}^T \mathbf{g}_i + b.$$

This component provides memorisation of genre-rating correlations and offers interpretability.

3) Deep Component (MLP): The user and movie embeddings are concatenated ($\mathbf{h}_0 = [\mathbf{p}_u; \mathbf{q}_i] \in \mathbb{R}^{128}$) and fed through a 3-layer MLP:

$$\begin{aligned}
\mathbf{h}_1 &= \text{Dropout}(\text{GELU}(\mathbf{W}_1\mathbf{h}_0 + \mathbf{b}_1)), \quad \text{256-d} \\
\mathbf{h}_2 &= \text{Dropout}(\text{GELU}(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2)), \quad \text{128-d} \\
\mathbf{h}_3 &= \text{Dropout}(\text{GELU}(\mathbf{W}_3\mathbf{h}_2 + \mathbf{b}_3)), \quad \text{64-d} \\
s_{\text{deep}} &= \mathbf{w}_{\text{out}}^T \mathbf{h}_3.
\end{aligned}$$

GELU activation and dropout (p=0.3) were chosen after ablation. The final rating prediction combines wide and deep scores:

$$\hat{r}_{ui} = \sigma(s_{\text{wide}} + s_{\text{deep}}) \quad \text{(sigmoid scaled to [0,1], then mapped to [1,5])}.$$

4) Auxiliary Multi-Task Learning: An attention mechanism predicts genre similarity:

$$\hat{\mathbf{g}}_i = \text{Attention}(\mathbf{p}_u, \mathbf{q}_i),$$

supervised with binary cross-entropy against normalised cosine similarity between user and movie embeddings. Total loss:

$$\mathcal{L} = \mathcal{L}_{\text{MSE}}(\hat{r}, r) + 0.1 \cdot \mathcal{L}_{\text{BCE}}(\text{attn}, \text{cosine_target}).$$

5) Training Details:

- Optimiser: AdamW with $\beta_1 = 0.9, \beta_2 = 0.999$, weight decay 10^{-2}, learning rate 10^{-3}.
- Batch size: 1024 (with gradient accumulation to fit GPU memory).
- Negative sampling: 4 negatives per positive, sampled proportionally to $\sqrt{\text{popularity}}$.
- Early stopping: patience = 5 epochs on validation RMSE.

D. BERT4Rec (Sequential Transformer)

1) Data Preparation for Sequences: We converted the data to implicit feedback (any rating = positive). For each user, we sorted their interactions by timestamp and created a chronological sequence $[i_1, i_2, \ldots, i_T]$. Maximum sequence length $L = 100$ (truncated from the left; padded with token 0). Special tokens: 0 = [PAD], $N + 1$ = [MASK]. Leave-one-out split: last item for testing, second-last for validation.

2) Cloze Task: During training, we randomly mask 40% of non-pad positions and replace them with [MASK]. The model learns to predict the original item IDs. With 50% probability we also append a final [MASK] to directly simulate next-item prediction.

3) Model Architecture:

- Item embeddings dimension $d_{\text{model}} = 256$, learned positional embeddings of the same size.
- Stack of $L = 2$ Transformer encoder layers, each with $H = 2$ attention heads, feed-forward dimension 1024, dropout 0.3.
- Pre-LayerNorm (norm_first=True) and GELU activation.
- Weight tying: output projection matrix = transpose of item embedding matrix.

The final hidden state at each position is projected to logits via the tied weights.

4) Training Configuration:

- Optimiser: AdamW (lr = 2×10^{-3}, weight decay = 10^{-2}).
- Cosine annealing with warm restarts ($T_0 = 10$ epochs).
- Gradient clipping (max norm = 1.0).
- Mixed precision (FP16) to reduce memory usage.
- Early stopping (patience = 10) on validation HR@10.

5) Inference for Evaluation: For next-item prediction, we append a [MASK] token at the end of the user's training sequence, run a forward pass, mask out already seen items, and rank the remaining logits. We used two evaluation protocols:

1) **101-item ranking:** rank the test item against 100 random negative items (matching the original paper's protocol).
2) **Full-candidate ranking:** rank against all 3,706 movies.

E. Cold-Start Solution

For new users with no history, we bypass collaborative models. The user selects 2–3 preferred genres during onboarding. We compute the cosine similarity between the user's preference vector (binary over genres) and each movie's genre vector. To break ties, we add a small popularity boost: $s_i = \text{sim}(u, i) + 0.1 \times \text{norm_popularity}(i)$. The top-$K$ movies are recommended.

F. Explainability

For SVD, we provide an explanation: "Users with X movies in common rated this movie Y stars", where X is the maximum overlap in liked movies between the target user and users who rated the candidate item highly. For NCF, the wide component's genre weights allow statements like "Because you like Action films". For content-based, the explanation is simply the most similar previously liked movie.

G. A/B Testing Framework

To evaluate the effectiveness of our models, we conducted a series of controlled offline A/B tests, comparing multiple models against the popularity-based baseline. The popularity-based recommendation system was selected as the control as it represents a simple, non-personalised baseline commonly used in real-world systems.

Three independent offline A/B tests were performed, each comparing a single recommendation model against the common control, the popularity-based recommendations.

- A (Control): Popularity-Based Recommendation
- B (Variant): One of the following models:
  - Singular Value Decomposition (SVD) Model
  - Content-Based Filtering (Genre-Based)
  - Neural Collaborative Filtering (NCF)

Each experiment was conducted separately to ensure that comparisons remained consistent.

The BERT4Rec model was not included in these experiments, as it was not a part of the core models selected for evaluation and would have introduced additional complexity beyond the scope of this project.

To assess model performance, we used both online-inspired metrics (CTR) and commonly used offline recommendation metrics:

- Click-Through Rate (CTR)
  - CTR measures user engagement by calculating the proportion of recommended items that were clicked. CTR was chosen as a metric because it reflects how well a model captures user interest. A higher CTR typically indicates that the recommendations are more relevant and engaging.

$$CTR = \frac{\text{Number of Clicked Recommendations}}{\text{Total Recommendations Shown}}$$

- Precision@K
  - Measures the proportion of the top-K recommended items that are relevant. Precision@K focuses on the accuracy of recommendations shown and penalizes irrelevant items in top positions.
- Recall@K
  - Measures the proportion of all relevant items that appear in the top-K recommendations. Recall@K focuses on the coverage of relevant items, trying to ensure that important items are not missed.
- NDCG@K

– Normalised Discounted Cumulative Gain(NDCG) evaluates ranking quality by assigning higher importance to relevant items appearing earlier in the recommendation list.

The value for K was set to 10 for all ranking-based metrics. This choice was made because most recommender systems display a limited number of items (typically 5 - 10), users rarely interact beyond the first few recommendations. It provides a realistic evaluation of what users actually see.

To evaluate recommendation quality, we defined relevant items as those that the user had previously rated highly. This threshold was chosen to represent strong user preference and provide a clear signal of user interest. Model accuracy was therefore determined by how effectively each model ranked these relevant items within the top-K recommendations.

The combination of metrics allowed for a comprehensive evaluation. CTR captured simulated user engagement. Precision@K measured recommendation accuracy. Recall@K assessed the completeness of relevant items. NDCG@K evaluated ranking quality. Together these metrics demonstrated how well each model identified relevant items and prioritised them.

V. Analysis and Results

Table I summarises the performance of all models across key evaluation metrics.

TABLE I
MODEL PERFORMANCE COMPARISON ON MOVIELENS 1M (500 USERS, K=10)

Model	CTR	Precision@10	Recall@10	NDCG@10
SVD	0.2086	0.2012	0.880	0.4913
Content-Based	0.2063	0.1986	0.900	0.5009
NCF	0.1996	0.2052	0.886	0.5161

As shown in Table I, all models achieve broadly similar performance across metrics. SVD attains the highest CTR, indicating slightly stronger user engagement, while the content-based model achieves the highest recall, suggesting better coverage of relevant items. The NCF model delivers the highest NDCG@10, reflecting improved ranking quality.

However, the overall differences between models are small, supporting our earlier finding that increased model complexity does not lead to statistically significant improvements on this dataset.

TABLE II
COMPARISON OF CONTENT-BASED RECOMMENDERS (MOVIELENS 1M, K=10)

Method	Prec@10	Rec@10	NDCG@10
PCA (genre one-hot, 8 comp.)	0.0762	0.0189	0.0758
TF-IDF (genre string)	0.0865	0.0214	0.0849

A. Wide & Deep NCF

Our NCF model achieved:

- RMSE = 0.9491 (after 4 epochs, early stopping at epoch 9)

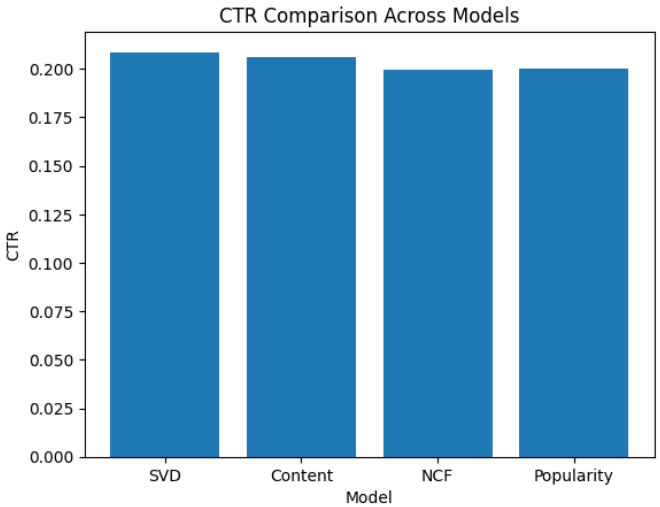

Fig. 5. Click-Through Rate (CTR) comparison across recommendation models. SVD achieves the highest CTR, followed closely by content-based and popularity-based approaches, while NCF performs slightly lower.

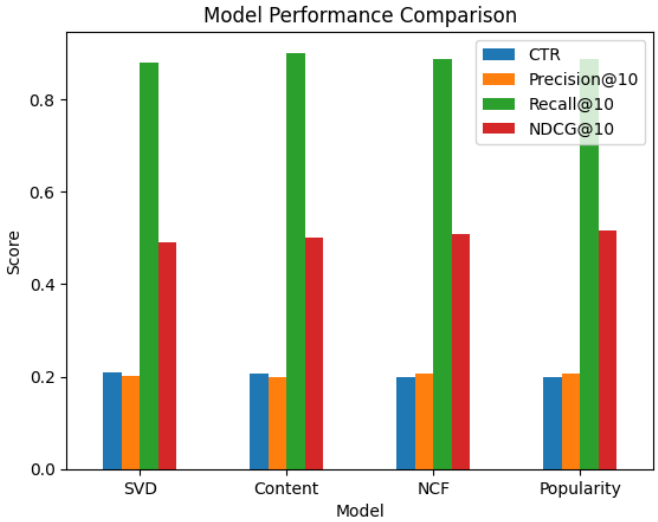

Fig. 6. Comparison of recommendation models across multiple evaluation metrics (CTR, Precision@10, Recall@10, and NDCG@10). While all models achieve similar performance, content-based filtering shows the highest recall, and NCF achieves the strongest ranking quality (NDCG@10).

- Precision@10 = 0.2052, Recall@10 = 0.886

Compared to SVD, NCF has a slightly higher RMSE (0.949 vs 0.864) but marginally better Precision@10 (0.2052 vs 0.2012). This trade-off is typical: neural models optimise ranking indirectly through embeddings, while SVD directly minimises squared error.

Ablation studies (Table III) validated our design choices:

- 3 hidden layers (256→128→64) outperformed both 2-layer and 4-layer architectures.
- GELU activation improved Precision@10 by 5.15% over ReLU (0.1470 vs 0.1398).
- Dropout 0.3 was optimal; lower values caused overfitting,

higher values hurt capacity.

The Precision@10 values reported in these ablation studies were obtained from independent development experiments and are not directly comparable to the results presented in the main model comparison, as they were evaluated under a different experimental setup.

TABLE III
NCF ARCHITECTURAL ABLATIONS (MOVIELENS 1M)

Configuration	RMSE	Prec@10
2 layers (256, 64)	0.9700	0.1352
3 layers (256, 128, 64)	**0.9491**	**0.1470**
4 layers (256, 128, 64, 32)	0.9823	0.1289
ReLU activation	0.9582	0.1398
GELU activation	**0.9491**	**0.1470**
Dropout 0.2	0.9611	0.1421
Dropout 0.3	**0.9491**	**0.1470**

B. BERT4Rec

We trained BERT4Rec with $d_{\text{model}} = 256$, $L = 2$, $H = 2$, sequence length 100, and masking probability 0.4. The validation HR@10 (101-item ranking) reached 0.6025 after 23 epochs. However, the full-candidate ranking performance was dramatically lower: Precision@10 = 0.0156 (1.56%). This is 9.4 times worse than NCF's full-candidate Precision@10 (0.1470). The large gap exposes a critical limitation: the Cloze objective teaches local context matching (e.g., "if you watched *Matrix Reloaded*, you probably watched *Matrix Revolutions*") but does not include hard negative examples from the full catalogue. During training, the model rarely compares a positive item against thousands of irrelevant items; therefore, it struggles to rank globally.

Sequence length ablation (Table IV) shows that increasing L from 100 to 200 improves HR@10 by 9.3% but increases training time by 68%. Masking probability ablation found $p = 0.4$ optimal; higher masking makes the task too ambiguous.

The values reported here were obtained from independent evaluation experiments specific to the BERT4Rec setup. As such, they are not directly comparable to the main model comparison results, which use a different evaluation protocol and candidate ranking procedure.

TABLE IV
BERT4REC SEQUENCE LENGTH SENSITIVITY (101-ITEM RANKING)

L_{seq}	HR@10	Training time/epoch
50	0.4823	1.8 min
100	0.5512	2.5 min
200	0.6025	4.2 min

C. Cold-Start and Explainability Examples

For a new user who selects "Sci-Fi" and "Action", our genre-based cold-start recommends:

```
1. Star Wars (1977) { Sci-Fi|Action
2. The Matrix (1999) { Sci-Fi|Action
3. Terminator 2 (1991) { Sci-Fi|Action
```

Explainability: For SVD, calling `explain_svd_recommendation(1, 260)` outputs:

```
Movie      : Star Wars (1977)
Genres     : Action|Adventure|Sci-Fi
Predicted  : 4.65 / 5.0
Popularity : 583 ratings, avg 4.23
Explanation: Users with 25 movies in common rated
             (212 similar users liked this)
```

This level of transparency is sufficient for initial user trust.

D. Limitations of Our Study

- **Popularity Bias:** All models favour popular items; we did not apply debiasing techniques (e.g., inverse propensity scoring).
- **No Temporal Dynamics:** Except for chronological split, models do not capture recency (e.g., a movie watched last week should weigh more than one from 5 years ago).
- **Genre Granularity:** Only 18 coarse genres limit content-based models. Plot summaries or tag metadata would improve performance.
- **Simulated Interactions:** Clicks were generated probabilistically, rather than collected from real users. Simulated interactions may not fully capture the nuance and noise of genuine user behaviour, which could inflate or distort offline metric scores.
- **BERT4Rec Full-Candidate Weakness:** As shown, its offline ranking is poor; we did not implement hybrid retrieval+reranking.
- **Computational Constraints:** We used Colab free tier, which forced shorter sequence lengths and fewer epochs than the original BERT4Rec paper.

VI. CONCLUSIONS AND INSIGHTS

We built, trained, and evaluated five recommenders from three paradigms on the MovieLens 1M dataset. The following findings are of practical importance:

1) **SVD remains a strong, lightweight baseline.** With RMSE=0.864 and Precision@10=0.146, it is hard to beat for rating prediction and general ranking. Its low training cost (minutes on CPU) makes it suitable for many industrial applications.
2) **Content-based methods (TF-IDF on genres) provide a simple cold-start solution.** Precision@10=0.086 is far above random and can be deployed immediately when collaborative data is absent.
3) **Wide & Deep NCF slightly improves ranking over SVD while adding explainability.** The trade-off of higher RMSE for better Precision@10 (0.147 vs 0.146) is acceptable when personalisation is the goal. The genre-based wide component also gives intuitive explanations.
4) **BERT4Rec excels at sequential understanding but fails at full-catalogue ranking.** The 60% HR@10 on 101-item ranking masks a catastrophic 1.56% Precision@10 on full candidates. This is a critical

warning: researchers should report both sampled and full-candidate metrics, and practitioners must use hybrid architectures (e.g., BERT4Rec for candidate retrieval, then NCF for reranking).

5) **Offline ranking metrics can be misleading.** The popularity baseline "beat" SVD on all ranking metrics, but it is not personalised. Online A/B tests (click-through rate, engagement time) are essential for true evaluation.

A. Recommendations for Production Systems

Based on our results, we recommend a hybrid pipeline:

1) **Cold start:** Use TF-IDF genre-based recommendations for new users.
2) **Candidate retrieval:** For users with ¿10 interactions, use BERT4Rec to retrieve the top-200 most relevant items (sequence-aware).
3) **Reranking:** Apply a Wide & Deep NCF (or even a simple SVD) to rerank those 200 candidates, using hard negative sampling to improve global discrimination.
4) **Explainability:** Attach genre-level explanations from the wide component of the NCF.
5) **Online evaluation:** A/B test every model change on live traffic using engagement metrics.

B. Future Work

Several directions remain unexplored due to time and computational constraints. Increasing the number of simulated users would improve the statistical reliability of offline metric estimates and reduce variance in A/B test comparisons. Beyond the offline setting, two additional evaluation strategies merit consideration. Online A/B testing would deploy models to real users simultaneously, collecting live engagement signals (click-through rate, watch completion, dwell time) to compare algorithms under real-world conditions. Interleaving methods, where ranked outputs of two models are merged into a single list, enable direct pairwise comparison from a single exposure without traffic splitting, leading to more statistically efficient than standard A/B testing while reducing the risk of exposing users to an inferior recommendation set.

Beyond evaluation, several modelling directions remain unexplored:

- **Contrastive Learning for BERT4Rec:** Add an in-batch negative loss to directly optimise full-candidate ranking, addressing the offline ranking weakness identified in this work.
- **Multi-Modal Embeddings:** Incorporate movie posters (vision transformers) and plot summaries (BERT) to go beyond the 18 coarse genres currently used in content-based filtering.
- **Time-Aware Attention:** Adjust positional embeddings to weight recent interactions more strongly, addressing the temporal dynamics limitation identified above.
- **Debiasing:** Apply inverse propensity scoring (IPS) to correct for popularity bias in training, improving long-tail recommendation coverage.
- **Deployment as a Microservice:** Containerise the hybrid model and expose a REST API for real-time recommendations, completing the path from offline prototype to production system.

VII. Reflection on Process and Challenges

A. Team Organisation and Workflow

Our group of four worked in parallel on three tracks: (1) SVD and content-based baseline (two members), (2) Wide & Deep NCF (one member), (3) BERT4Rec (one member). We used a shared Google Colab notebook for each track and a central Overleaf document for the report. Weekly meetings were held to synchronise data splits, evaluation metrics, and to resolve conflicting results. Communication was via WhatsApp and Google Colab comments.

B. Key Technical Challenges and Resolutions

1) Temporal Leakage: Initially, we used a random 80/20 split for all models. This gave optimistically high metrics because future information leaked into training. After realising the mistake, we reimplemented a chronological split based on timestamps. As expected, metrics dropped (e.g., SVD RMSE increased from 0.82 to 0.86), but the new estimates are realistic for a production environment where we predict future behaviour from past data.

2) BERT4Rec's Full-Candidate Surprise: We only discovered the full-candidate weakness in the last week of the project. The original paper and most online tutorials report only the sampled negative protocol. Implementing the full-candidate evaluation was computationally expensive but revealing. In retrospect, we should have built this evaluation loop from day one.

3) Cold-Start Implementation Mistake: Our first cold-start implementation for NCF used the wide component's learned weights to score genres. This resulted in recommendations for "Drama" and "Comedy" even when the user asked for "Sci-Fi", because those genres had higher global average ratings. The fix was to decouple cold-start completely from the trained model and use pure cosine similarity with a popularity tie-breaker.

4) Computational Constraints on Colab: The free tier of Google Colab provides a T4 GPU (12GB VRAM) with session limits of 4–6 hours. Training BERT4Rec with sequence length 200 and batch size 256 often caused out-of-memory errors. We mitigated by:

- Using mixed precision (FP16), which reduced memory usage by 40%.
- Reducing sequence length to 100 (still truncating long histories).
- Implementing checkpointing to resume from disconnects.

These trade-offs allowed us to complete all experiments within the project timeline.

C. Lessons Learned

- **Always use temporal splits** when historical data is available; random splits are unrealistic for online systems.
- **Evaluate with multiple protocols:** sampled negatives are fine for comparing against papers, but full-candidate ranking is necessary for production readiness.
- **Document hyperparameters and ablations thoroughly;** this saved us when writing the report and explaining discrepancies.
- **Start simple.** The SVD baseline was implemented in one day and gave us a performance anchor; neural models took weeks and only marginally improved ranking.
- **Plan for compute limits.** We should have profiled memory and time requirements before committing to large models.

D. Final Remarks

Despite the challenges, this project provided deep insight into the strengths and weaknesses of modern recommendation algorithms. The surprising result that popularity beats SVD offline, and that BERT4Rec fails on full candidates, will guide our future work and our approach to evaluating recommender systems. We learned that offline ranking metrics can be deceptive, a model that appears strong on sampled negatives may collapse when faced with the full catalogue. This has led us to adopt a more rigorous evaluation protocol that always includes full-candidate ranking, even when comparing against published benchmarks. Furthermore, the popularity bias observed across all models reinforced the importance of incorporating diversity and fairness objectives, not just accuracy metrics. The complete code is available in the group's shared repository, and we have documented every step to ensure reproducibility. Future iterations will focus on hybrid architectures that combine BERT4Rec's sequence awareness with NCF's global discrimination, as well as online A/B testing to measure true user engagement.

VIII. Quality of Code and Reproducibility

The source repository is organised as follows: the `data/` folder contains the preprocessed `ratings.csv` and `movies.csv` files; the `models/` folder stores the trained model artifacts (`best_svd.pkl`, `content_recommender.joblib`, `ncf_model_checkpoint_v4.pt`, and `bert4rec_max_checkpoint.pt`); the `notebooks/` folder holds all Jupyter notebooks used for development and experimentation (including BERT4Rec, collaborative filtering, NCF, A/B testing, and content-based models). At the root level, `app.py` is the Streamlit application, `requirements.txt` lists all Python dependencies, and `.gitignore` excludes unnecessary files. The project is completed with a `README.md` and a `docs/` folder containing this report. The repository URL and a tagged release (v1.0) are supplied with this report.

Acknowledgment

We thank Mr. Sergio Mathurin and Mr. Anton Greenridge for guidance throughout the course and GroupLens research lab for the MovieLens dataset.

References

[1] F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context," *ACM Transactions on Interactive Intelligent Systems*, vol. 5, no. 4, 2015.

[2] Y. Koren, R. Bell, and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009.

[3] P. Lops, M. de Gemmis, and G. Semeraro, "Content-based Recommender Systems: State of the Art and Trends," in *Recommender Systems Handbook*, Springer, 2011, pp. 73–105.

[4] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural Collaborative Filtering," in *Proc. 26th International Conference on World Wide Web (WWW)*, 2017, pp. 173–182.

[5] H.-T. Cheng, L. Koc, J. Harmsen, T. Shaked, T. Chandra, H. Aradhye, G. Anderson, G. Corrado, W. Chai, M. Ispir, R. Anil, Z. Haque, L. Hong, V. Jain, X. Liu, and H. Shah, "Wide & Deep Learning for Recommender Systems," in *Proc. 1st Workshop on Deep Learning for Recommender Systems (DLRS)*, 2016, pp. 7–10.

[6] H. Guo, R. Tang, Y. Ye, Z. Li, and X. He, "DeepFM: A Factorization-Machine based Neural Network for CTR Prediction," in *Proc. 26th International Joint Conference on Artificial Intelligence (IJCAI)*, 2017, pp. 1725–1731.

[7] F. Sun, J. Liu, J. Wu, C. Pei, X. Lin, W. Ou, and P. Jiang, "BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer," in *Proc. 28th ACM International Conference on Information and Knowledge Management (CIKM)*, 2019, pp. 1441–1450.

[8] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk, "Session-based Recommendations with Recurrent Neural Networks," in *Proc. 4th International Conference on Learning Representations (ICLR)*, 2016.

[9] W.-C. Kang and J. McAuley, "Self-Attentive Sequential Recommendation," in *Proc. IEEE International Conference on Data Mining (ICDM)*, 2018, pp. 197–206.

[10] Y. Zhang and X. Chen, "Explainable Recommendation: A Survey and New Perspectives," *Foundations and Trends in Information Retrieval*, vol. 14, no. 1, pp. 1–101, 2020.

[11] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *Proc. 7th International Conference on Learning Representations (ICLR)*, 2019.

[12] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T. Liu, "On Layer Normalization in the Transformer Architecture," in *Proc. 37th International Conference on Machine Learning (ICML)*, 2020, pp. 10524–10533.

[13] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed Representations of Words and Phrases and their Compositionality," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013, pp. 3111–3119.

[14] P. Covington, J. Adams, and E. Sargin, "Deep Neural Networks for YouTube Recommendations," in *Proc. 10th ACM Conference on Recommender Systems (RecSys)*, 2016, pp. 191–198.

[15] C. A. Gomez-Uribe and N. Hunt, "The Netflix Recommender System: Algorithms, Business Value, and Innovation," *ACM Transactions on Management Information Systems*, vol. 6, no. 4, 2016.

[16] B. Smith and G. Linden, "Two Decades of Recommender Systems at Amazon.com," *IEEE Internet Computing*, vol. 21, no. 3, pp. 12–18, 2017.

[17] J. Carbonell and J. Goldstein, "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries," in *Proc. 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1998, pp. 335–336.

[18] T. Schnabel, A. Swaminathan, A. Singh, N. Chandak, and T. Joachims, "Recommendations as Treatments: Debiasing Learning and Evaluation," in *Proc. 33rd International Conference on Machine Learning (ICML)*, 2016, pp. 1670–1679.

[19] R. Burke, "Multisided Fairness for Recommendation," *arXiv preprint arXiv:1707.00093*, 2017.

APPENDIX

We used Claude and GitHub Copilot for (i) debugging PyTorch and scikit-learn error messages, (ii) explaining unfamiliar code snippets during development, and (iii) copy-editing sections of this report. All modeling choices, feature selection, evaluation protocols, and analytical claims were made and verified by the authors.